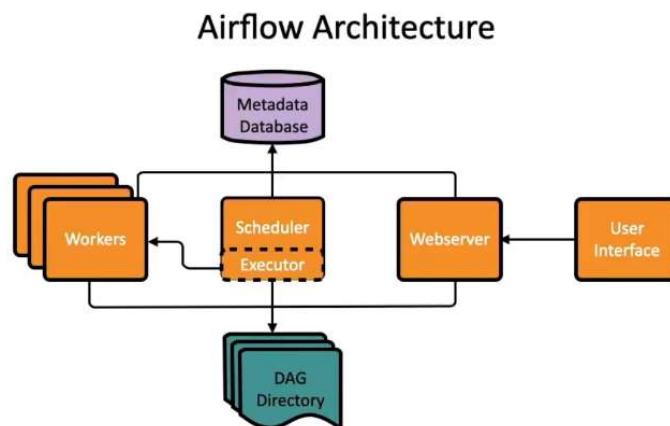


Airflow Assessment 1

Part 2: Conceptual Write-Up

What is Apache Airflow, and how does it work?

Apache Airflow is an **open-source workflow orchestration tool** used for scheduling, monitoring, and managing complex data pipelines. It allows developers to define workflows as Directed Acyclic Graphs (DAGs), where each node represents a task, and edges define dependencies between tasks. Workflows are written in Python, which makes Airflow flexible and highly extensible. Once a DAG is created, Airflow's **scheduler** ensures that tasks are executed in the correct order, respecting dependencies, and the **web UI** provides visibility into the pipeline status, logs, and progress.



Airflow in modern data engineering workflows:

In today's data-driven organizations, teams deal with multiple systems such as databases, cloud storage, APIs, and machine learning models. These systems need to be connected in a reliable sequence, often with dependencies. For example, a typical pipeline might extract data from a source system, transform it into a usable format, and load it into a data warehouse for analytics. Airflow fits perfectly here by providing **automation, reliability, and monitoring**. Instead of manually running scripts, engineers define the workflow once in Airflow, and it automatically ensures timely execution, retries in case of failure, and logging for audits.

Airflow is also cloud-agnostic and integrates well with popular platforms like AWS, GCP, and Azure, making it a common choice in modern data engineering stacks.

Airflow vs. traditional schedulers or similar tools:

Unlike traditional schedulers such as **cron jobs**, Airflow provides task dependency management, visualization, and monitoring in a web UI. Cron is limited to scheduling commands at fixed times, but Airflow allows for **dynamic, event-driven, and dependency-based workflows**.

Compared to other orchestration tools like **Luigi** or **Prefect**, Airflow is more widely adopted and mature. Luigi is lightweight but less feature-rich in terms of UI and monitoring. Prefect is newer and emphasizes simplicity, but Airflow has stronger community support and integrations. In summary, Airflow strikes a balance between **flexibility, scalability, and visibility**, making it suitable for enterprise-scale pipelines.

Key components of Airflow:

- **DAG (Directed Acyclic Graph):** Represents the workflow structure, showing the sequence of tasks.
- **Operators:** Templates that define what each task does. Examples include Python Operator for Python functions and Bash Operator for shell commands.
- **Scheduler:** Determines when and how tasks should run, based on DAG definitions.
- **Executor:** Handles the actual task execution. Depending on the setup, executors can run tasks locally, in parallel, or on distributed clusters.
- **Web UI:** Provides monitoring, visualization of DAGs, and access to logs for debugging.

These components interact in a loop: the **DAG defines tasks** → **Scheduler schedules them** → **Executor runs them** → **UI shows progress and logs**.

Real-world use cases:

Airflow is widely used in enterprises for **ETL pipelines, data warehousing, reporting, and machine learning workflows**. For instance:

- Automating daily extraction of sales data, transforming it, and loading it into a BI system.
- Orchestrating model training pipelines in machine learning.
- Managing cloud data migrations and API-based workflows.

Because of its **scalability and visibility**, Airflow is particularly useful in scenarios where data workflows are **complex, recurring, and business-critical**.